

UNIT – I:

Introduction to Algorithm Analysis, Space and Time Complexity analysis, Asymptotic Notations.

AVL Trees – Creation, Insertion, Deletion operations and Applications.

B-Trees – Creation, Insertion, Deletion operations and Applications.

1. Introduction to algorithm analysis

An Algorithm is a finite set of instructions that, if followed, accomplishes a particular task. In addition, all algorithms should satisfy the following criteria.

- **INPUT** □ Zero or more quantities are externally supplied.
- **OUTPUT** □ At least one quantity is produced.
- **DEFINITENESS** □ Each instruction is clear and unambiguous.
- **FINITENESS** □ If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
- **EFFECTIVENESS** □ Every instruction must be very basic so that it can be carried out in principle, by a person using only pencil & paper.

Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. Analysis of algorithms is the determination of the amount of time and space resources required to execute it.

1.1. Why Analysis of Algorithms is important?

- To predict the behavior of an algorithm without implementing it on a specific computer.
- It is much more convenient to have simple measures for the efficiency of an algorithm than to implement the algorithm and test the efficiency every time a certain parameter in the underlying computer system changes.
- It is impossible to predict the exact behavior of an algorithm. There are too many influencing factors.
- The analysis is thus only an approximation; it is not perfect.
- More importantly, by analyzing different algorithms, we can compare them to determine the best one for our purpose.

1.2. Types of Algorithm Analysis:

1. Best case
2. Worst case
3. Average case

- **Best case:** Define the input for which algorithm takes less time or minimum time. In the best case calculate the lower bound of an algorithm. Example: In the linear search when search data is present at the first location of large data then the best case occurs.
- **Worst Case:** Define the input for which algorithm takes a long time or maximum time. In the worst calculate the upper bound of an algorithm. Example: In the linear search when search data is not present at all then the worst case occurs.
- **Average case:** In the average case take all random inputs and calculate the computation time for all inputs. And then we divide it by the total number of inputs.

Average case = all random case time / total no of case

2.SPACE AND TIME COMPLEXITY ANALYSIS:

1. Space Complexity:

The space complexity of an algorithm is the amount of memory it needs to run to compilation.

2. Time Complexity:

The time complexity of an algorithm is the amount of computer time it needs to run to compilation.

2.1.Space Complexity:

Space Complexity Example:

```
Algorithm abc(a,b,c)
{
  return a+b++*c+(a+b-c)/(a+b) +4.0;
}
```

→ The Space needed by each of these algorithms is seen to be the sum of the following component.

1.**A fixed part** that is independent of the characteristics (eg:number,size)of the inputs and outputs.

The part typically includes the instruction space (ie. Space for the code), space for simple variable and fixed-size component variables (also called aggregate) space for constants, and so on

2.**variable part** that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables (to the extent that it depends on instance characteristics), and the recursion stack

space.

a. The space requirement $s(p)$ of any algorithm p may therefore be written as,

$S(P) = c + S_p(\text{Instance characteristics})$ Where 'c' is a constant.

Example : Algorithm $\text{sum}(a,n)$

```
{
    s=0.0;
    for I=1 to n do
        s= s+a[I];
    return s;
}
```

- The problem instances for this algorithm are characterized by n , the number of elements to be summed. The space needed by 'n' is one word, since it is of type integer.
- The space needed by 'a' is the space needed by variables of type array of floating point numbers.
- This is at least 'n' words, since 'a' must be large enough to hold the 'n' elements to be summed.
- So, we obtain $S_{\text{sum}}(n) \geq (n+s)$ [n for a[], one each for n, I a & s]

2.2 Time Complexity:

The time $T(p)$ taken by a program P is the sum of the compile time and the run time (execution time)

→ The compile time does not depend on the instance characteristics. Also we may assume that a compiled program will be run several times without recompilation. This run time is denoted by $t_p(\text{instance characteristics})$.

→ The number of steps any problem statement is assigned depends on the kind of statement. For example, comments → 0 steps.

Assignment statements → 1 steps. [Which does not involve any calls to other algorithms]

Interactive statement such as for, while & repeat-until → Control part of the statement.

1. **We introduce a variable, count into the program statement to increment count with initial value 0. Statement to increment count by the appropriate amount are introduced into the program.**

This is done so that each time a statement in the original program is executed count is incremented by the step count of that statement.

Algorithm:

Algorithm sum(a,n)

```
{
  s= 0.0;
  count = count+1;
  for I=1 to n do
    count =count+1;
    s=s+a[I];
    count=count+1;
  }
  count=count+1;
  count=count+1;
  return s;
}
```

→ If the count is zero to start with, then it will be $2n+3$ on termination. So each invocation of sum execute a total of $2n+3$ steps.

2. The second method to determine the step count of an algorithm is to build a table in which we list the total number of steps contributed by each statement.

→ First determine the number of steps per execution (s/e) of the statement and the total number of times (ie., frequency) each statement is executed.

→ By combining these two quantities, the total contribution of all statements, the step count for the entire algorithm is obtained.

Statement	S/e	Frequency	Total
1. Algorithm Sum(a,n)	0	-	0
2. {	0	-	0
3. S=0.0;	1	1	1
4. for I=1 to n do	1	n+1	n+1
5. s=s+a[I];	1	n	n
6. return s;	1	1	1
7. }	0	-	0
Total			$2n+3$

3.ASYMPTOTIC NOTATIONS

There are different kinds of mathematical notations used to represent time complexity.

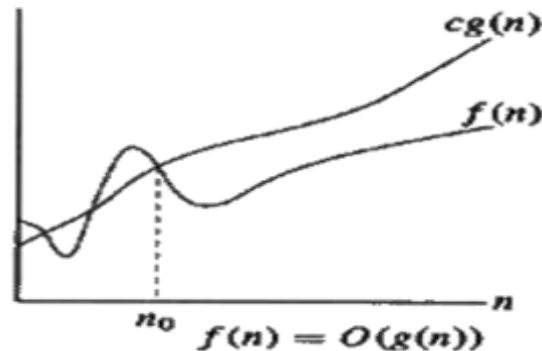
These are called asymptotic notations.

They are as follows:

1. Big oh(O) notation
2. Omega(Ω) notation
3. Theta(Θ) notation

3.1.Big oh(O) notation:

- Big oh(O) notation is used to represent upperbound of algorithm runtime.
- Let $f(n)$ and $g(n)$ are two non-negative functions
- The function $f(n) = O(g(n))$ if and only if there exists positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n, n \geq n_0$.



Example:

If $f(n)=3n+2$ then prove that $f(n) = O(n)$

Let $f(n) = 3n+2$, $c=4$, $g(n) = n$ if $n=1$ $3n+2 \leq 4n$

$$3(1)+2 \leq 4(1)$$

$$3+2 \leq 4$$

$$5 \leq 4 \text{ (F)}$$

if $n=2$ $3n+2 \leq 4n$ $3(2)+2 \leq 4(2)$

$$8 \leq 8 \text{ (T)}$$

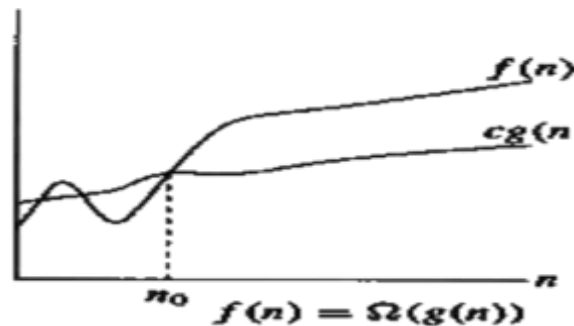
$$3n+2 \leq 4n \text{ for all } n \geq 2$$

This is in the form of $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$, where $c=4$, $n_0=2$ Therefore, $f(n) = O(n)$,

3.2.Omega(Ω) notation:

- Omega(Ω) notation is used to represent lowerbound of algorithm runtime.

- Let $f(n)$ and $g(n)$ are two non-negative functions
- The function $f(n) = \Omega(g(n))$ if and only if there exists positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n, n \geq n_0$.



Example

$f(n)=3n+2$ then prove that $f(n) = \Omega(g(n))$

Let $f(n) = 3n+2$, $c=3$, $g(n) = n$ if $n=1$ $3n+2 \geq 3n$

$$3(1)+2 \geq 3(1)$$

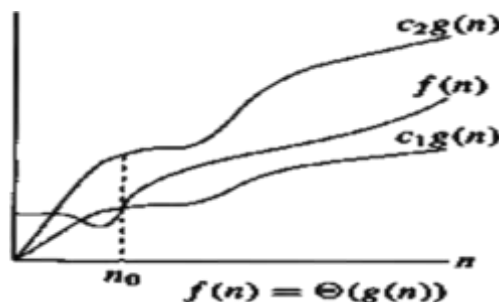
$$5 \geq 3 \text{ (T)}$$

$$3n+2 \geq 4n \text{ for all } n \geq 1$$

This is in the form of $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$, where $c=3$, $n_0=1$ Therefore, $f(n) = \Omega(n)$.

3.3.Theta(θ) notation:

- Theta(θ) notation is used to represent the running time between upper bound and lower bound.
- Let $f(n)$ and $g(n)$ be two non-negative functions.
- The function $f(n) = \theta(g(n))$ if and only if there exists positive constants c_1 , c_2 and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n, n \geq n_0$.



Example:

$f(n)=3n+2$ then Prove that $f(n) = \theta(g(n))$

Lower bound $= 3n+2 \geq 3n$ for all $n \geq 1$

$$c_1=3, g(n)=n, n_0=1$$

Upper Bound = $3n+2 \leq 4n$ for all $n \geq 2$

$$c_2=4, g(n)=n, n_0=2$$

$$3(n) \leq 3n+2 \leq 4(n) \text{ for all } n, n \geq 2$$

This is in the form of $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$

Where $c_1=3, c_2=4, g(n)=n, n_0=2$

Therefore $f(n) = \theta(n)$

3.4.LITTLE OH NOTATION

- Little-o ($o()$) notation is used to describe an upper bound that cannot be tight.
- little $o()$ means **loose upper-bound** of $f(n)$.
- Let $f(n)$ and $g(n)$ be two non-negative functions.
- In mathematical relation, $f(n) = o(g(n))$ means $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$
- Little o is a rough estimate of the maximum order of growth whereas Big-O may be the actual order of growth.

Definition: Let $f(n)$ and $g(n)$ be functions that map positive integers to positive real numbers. We say that $f(n)$ is $o(g(n))$ (or $f(n) \in o(g(n))$) if for **any real** constant $c > 0$, there exists an integer constant $n_0 \geq 1$ such that $0 \leq f(n) < c * g(n)$

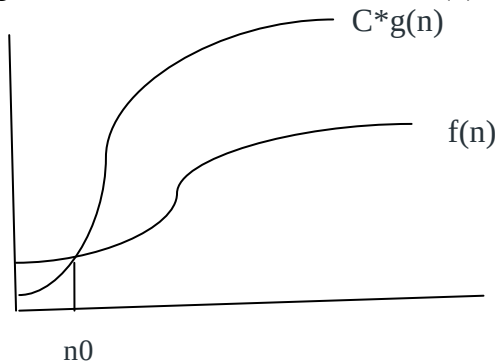


Fig: $f(n) = o(g(n))$

Examples:

Is $7n + 8 \in o(n^2)$?

In order for that to be true, for any c , we have to be able to find an n_0 that makes

$f(n) < c * g(n)$ asymptotically true.

lets took some example,

If $c = 100$, we check the inequality is clearly true. If $c = 1/100$, we'll have

Little o notation is used to describe an upper bound that cannot be tight. In other words, loose upper bound of $f(n)$.

Let $f(n)$ and $g(n)$ are the functions that map positive real numbers. We can say that the function $f(n)$ is $o(g(n))$ if for any real positive constant c , there exists an integer constant $n_0 \leq 1$ such that $f(n) > 0$.

4.AVL TREES

AVL tree is a height-balanced binary search tree. That means, an AVL tree is also a binary search tree but it is a balanced tree.

A binary tree is said to be balanced if, the difference between the heights of left and right subtrees of every node in the tree is either -1, 0 or +1.

In other words, a binary tree is said to be balanced if the height of left and right children of every node differ by either -1, 0 or +1.

In an AVL tree, every node maintains an extra information known as **balance factor**.

The AVL tree was introduced in the year 1962 by G.M. Adelson-Velsky and E.M. Landis.

An AVL tree is defined as follows...

The balance factor of a node is calculated either **height of left subtree - height of right subtree** (OR) **height of right subtree - height of left subtree**.

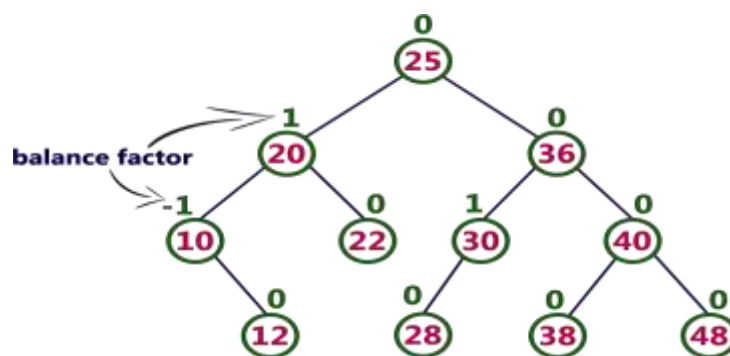
In the following explanation, we calculate as follows...

Balance factor = height Of LeftSubtree – height Of RightSubtree

Every AVL Tree is a binary search tree but every Binary Search Tree need not be AVL tree.

Example of AVL Tree

The above tree is a binary search tree and every node is satisfying balance factor condition. So this tree is said to be an AVL tree.



4.1.AVL Tree Rotations

In AVL tree, after performing operations like insertion and deletion we need to check the **balance factor** of every node in the tree.

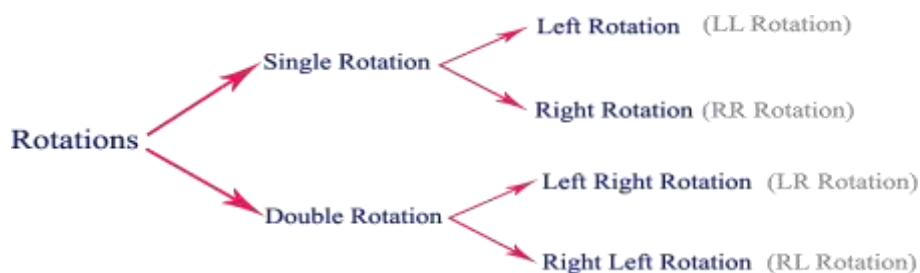
If every node satisfies the balance factor condition then we conclude the operation otherwise we must make it balanced.

Whenever the tree becomes imbalanced due to any operation we use **rotation** operations to make the tree balanced.

Rotation operations are used to make the tree balanced.

Rotation is the process of moving nodes either to left or to right to make the tree balanced.

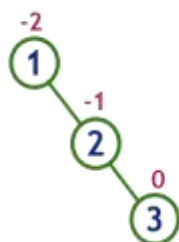
There are **four** rotations and they are classified into **two** types.



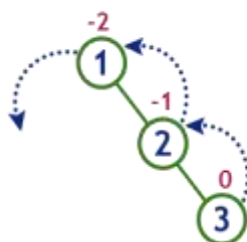
i) Single Left Rotation (LL Rotation)

In LL Rotation, every node moves one position to left from the current position. To understand LL Rotation, let us consider the following insertion operation in AVL Tree...

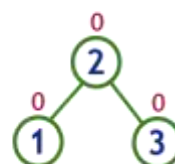
insert 1, 2 and 3



Tree is imbalanced



To make balanced we use LL Rotation which moves nodes one position to left

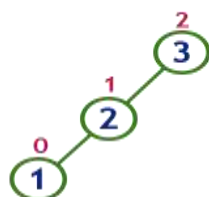


After LL Rotation Tree is Balanced

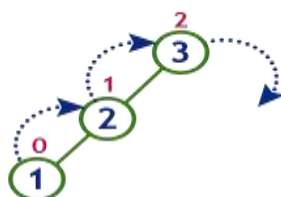
ii) Single Right Rotation (RR Rotation)

In RR Rotation, every node moves one position to right from the current position. To understand RR Rotation, let us consider the following insertion operation in AVL Tree...

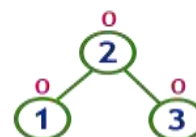
insert 3, 2 and 1



Tree is imbalanced
because node 3 has balance factor 2



To make balanced we use RR Rotation which moves nodes one position to right

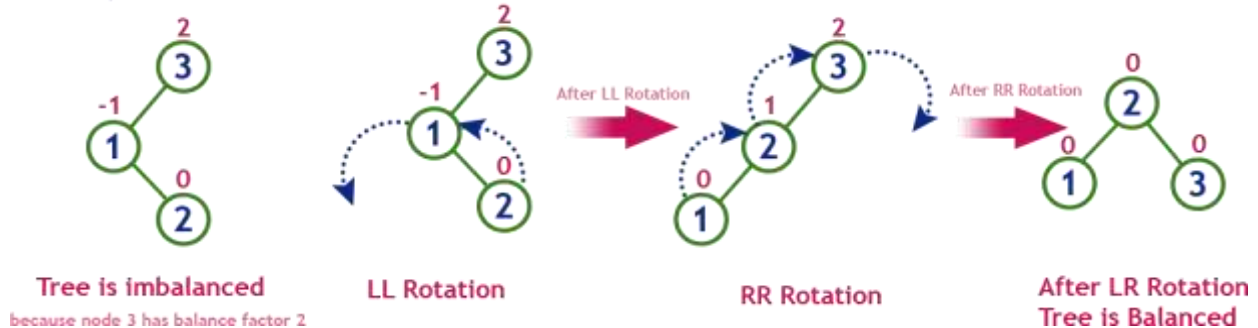


After RR Rotation Tree is Balanced

iii) Left Right Rotation (LR Rotation)

The LR Rotation is a sequence of single left rotation followed by a single right rotation. In LR Rotation, at first, every node moves one position to the left and one position to right from the current position. To understand LR Rotation, let us consider the following insertion operation in AVL Tree...

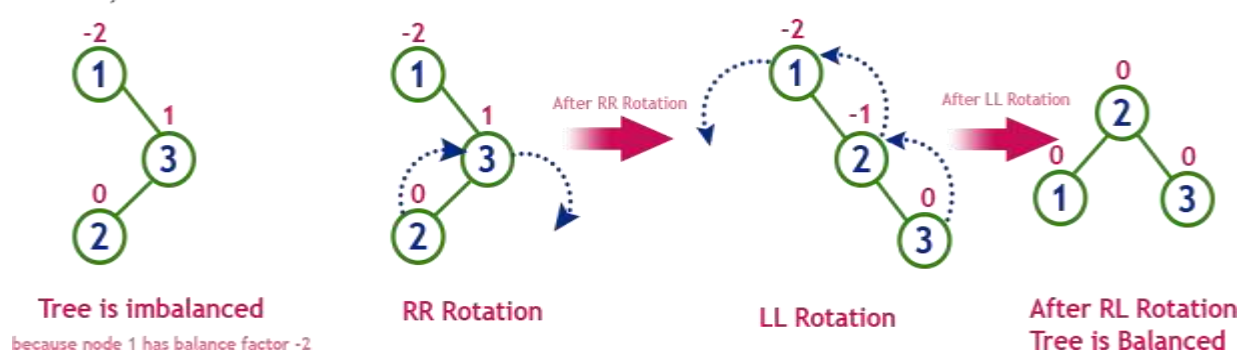
insert 3, 1 and 2



iv) Right Left Rotation (RL Rotation)

The RL Rotation is sequence of single right rotation followed by single left rotation. In RL Rotation, at first every node moves one position to right and one position to left from the current position. To understand RL Rotation, let us consider the following insertion operation in AVL Tree...

insert 1, 3 and 2



4.2. Operations on an AVL Tree

The following operations are performed on AVL tree...

1. Creation
2. Insertion
3. Deletion

4.2.1.Creation Operation in AVL Tree

Steps to Create an AVL Tree

Step 1: Initialize the AVL Tree

- Begin by initializing the AVL Tree with a root node set to null.

Step 2: Insert Nodes

- For each node to be inserted:

1.Binary Search Tree Insertion:

Insert the node following the standard binary search tree (BST) insertion procedure:

- Compare the key with the root.
- Recursively move to the left child if the key is less, or the right child if greater.
- Insert the node in the appropriate position when a null child is found.

2. Update Height:

- Update the height of each node visited during the insertion by recalculating it based on the heights of its children.

3. Calculate Balance Factor:

- Compute the balance factor of the node to check if the tree has become unbalanced (balance factor not in the range [-1, 1]).

struct node

```
{  
    int data;  
    struct node* left;  
    struct node* right;  
}
```

4.2.2.Insertion Operation in AVL Tree

In an AVL tree, the insertion operation is performed with **O(log n)** time complexity. In AVL Tree, a new node is always inserted as a leaf node. The insertion operation is performed as follows...

- **Step 1** - Insert the new element into the tree using Binary Search Tree insertion logic.
- **Step 2** - After insertion, check the **Balance Factor** of every node.

- **Step 3** - If the **Balance Factor** of every node is **0 or 1 or -1** then go for next operation.
- **Step 4** - If the **Balance Factor** of any node is other than **0 or 1 or -1** then that tree is said to be imbalanced. In this case, perform suitable **Rotation** to make it balanced and go for next operation

Example: Construct an AVL Tree by inserting numbers from 1 to 8.

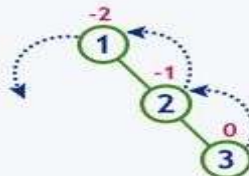
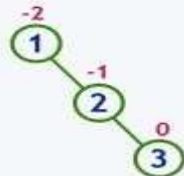
insert 1



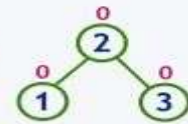
insert 2



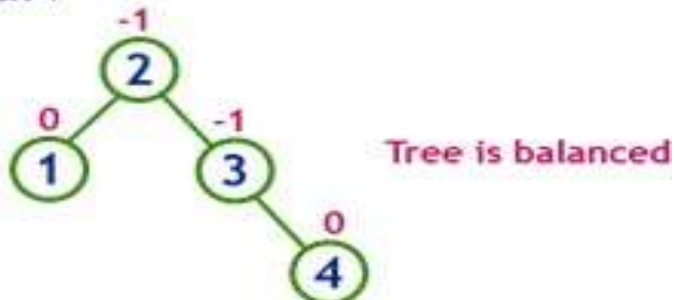
insert 3

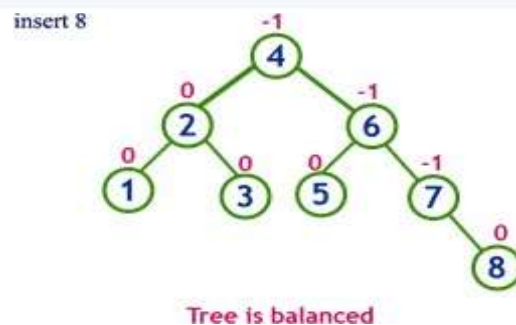
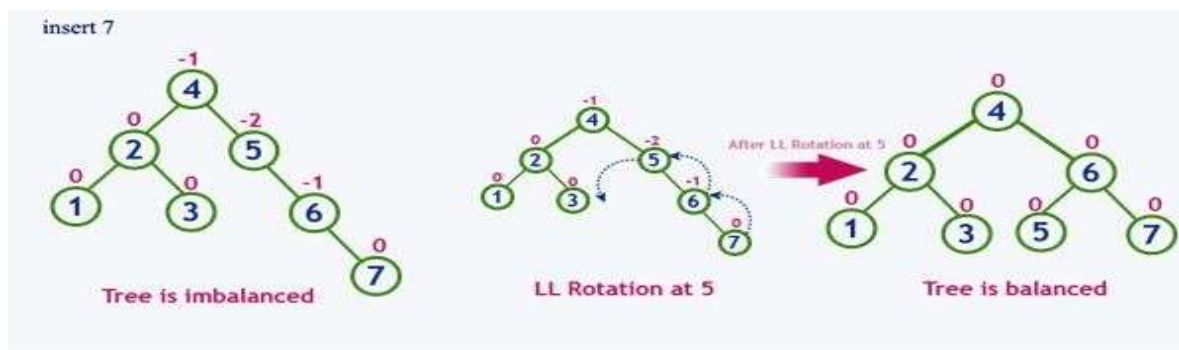
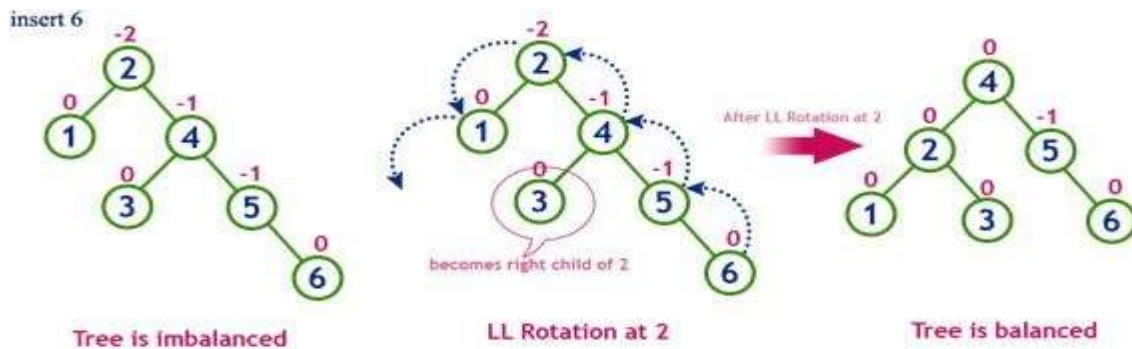
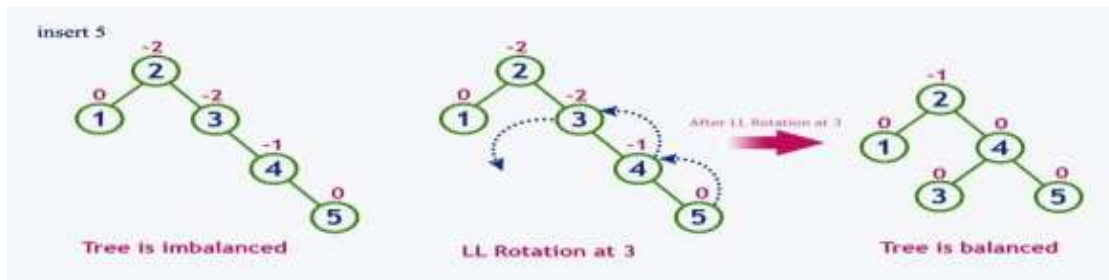


After LL Rotation



insert 4





4.2.3. Deletion Operation in AVL Tree

The deletion operation in AVL Tree is similar to deletion operation in BST. But after every deletion operation, we need to check with the Balance Factor condition. If the tree is balanced after deletion go for next operation otherwise perform suitable rotation to make the tree Balanced.

The two types of rotations are **L rotation** and **R rotation**. Here, we will discuss R rotations. L rotations are the mirror images of them.

If the node which is to be deleted is present in the left sub-tree of the critical node, then L rotation needs to be applied else if, the node which is to be deleted is present in the right sub-tree of the critical node, the R rotation will be applied.

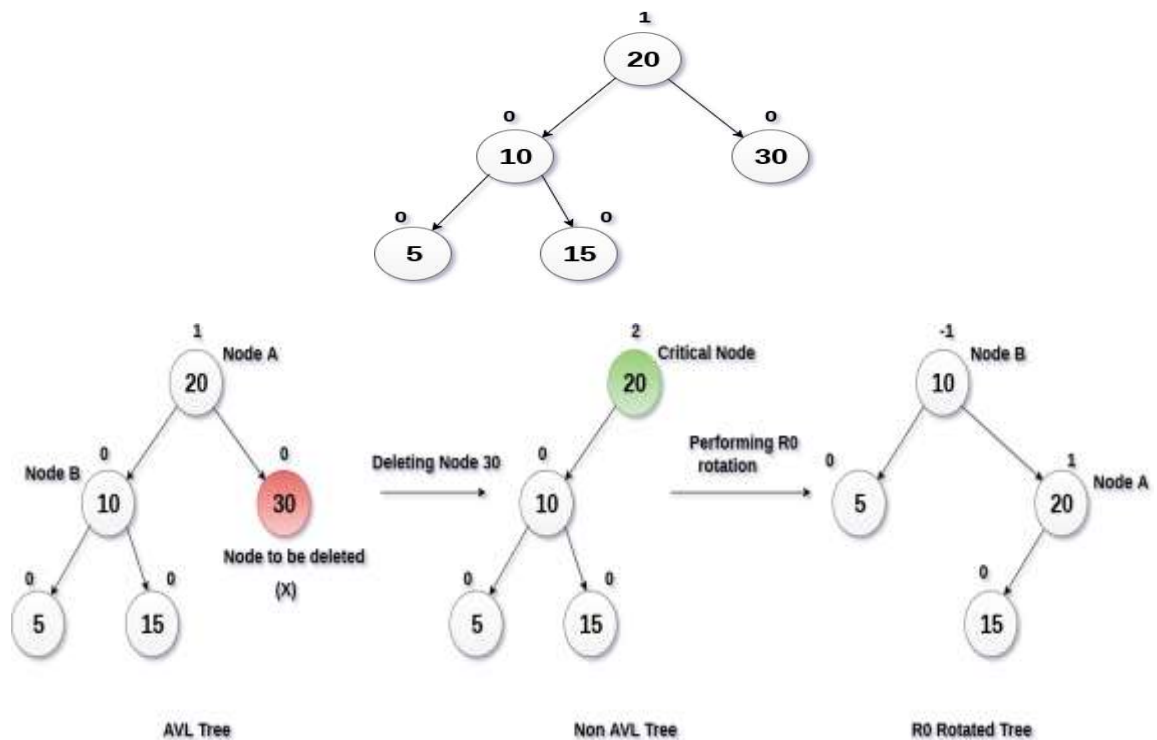
Let us consider that, A is the critical node and B is the root node of its left sub-tree. If node X, present in the right sub-tree of A, is to be deleted, then there can be three different situations:

a) R0 rotation (Node B has balance factor 0)

If the node B has 0 balance factor, and the balance factor of node A disturbed upon deleting the node X, then the tree will be rebalanced by rotating tree using R0 rotation.

Example:

Delete the node 30 from the AVL tree shown in the following image.

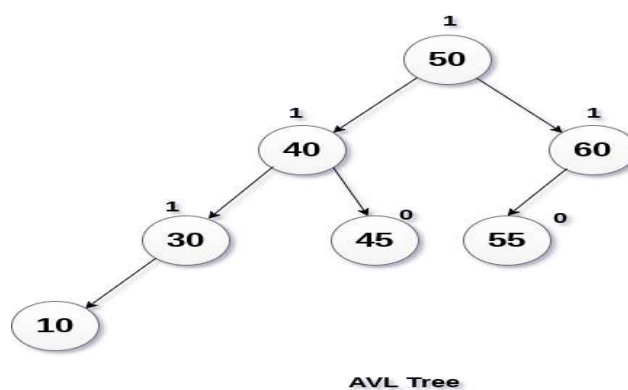


b) R1 Rotation (Node B has balance factor 1)

R1 Rotation is to be performed if the balance factor of Node B is 1.

Example

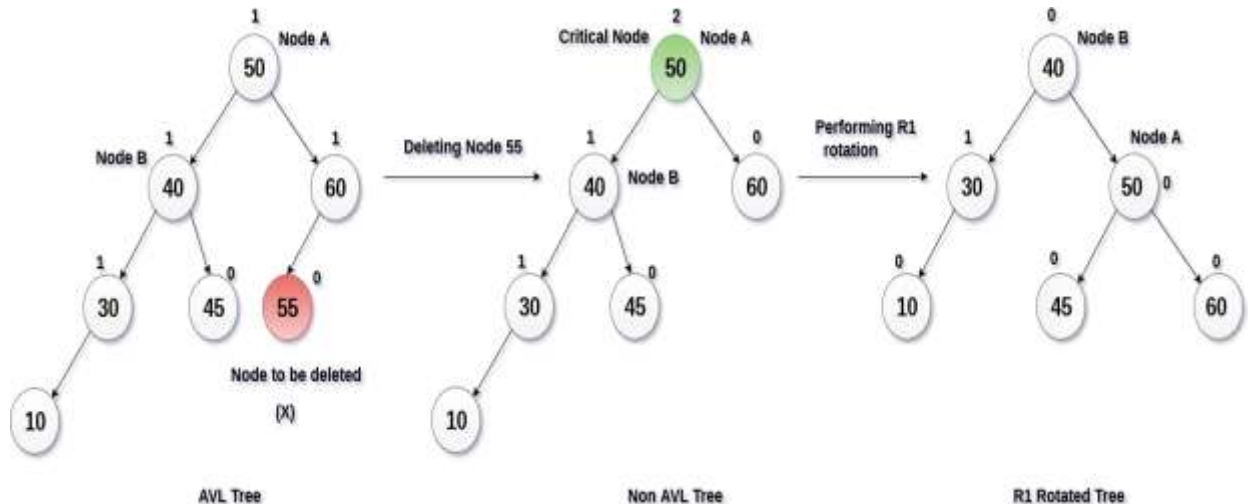
Delete Node 55 from the AVL tree shown in the following image.



Solution :

Deleting 55 from the AVL Tree disturbs the balance factor of the node 50 i.e. node A which becomes the critical node. This is the condition of R1 rotation in which, the node A will be moved to its right (shown in the image below). The right of B is now become the left of A (i.e. 45).

The process involved in the solution is shown in the following image.

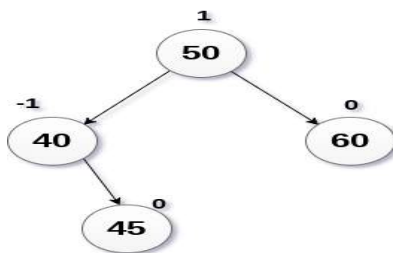


c)R-1 Rotation (Node B has balance factor -1)

R-1 rotation is to be performed if the node B has balance factor -1. This case is treated in the same way as LR rotation.

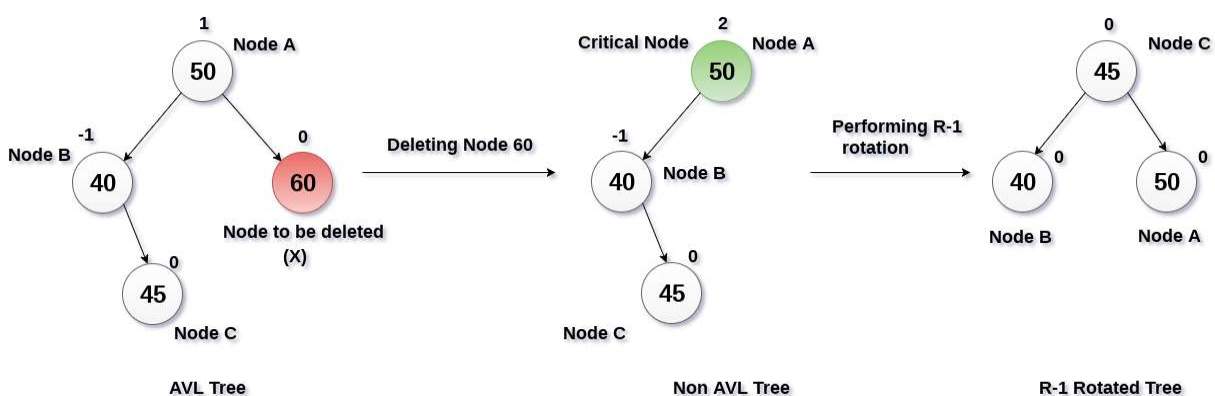
Example

Delete the node 60 from the AVL tree shown in the following image.



Solution:

in this case, node B has balance factor -1. Deleting the node 60, disturbs the balance factor of the node 50 therefore, it needs to be R-1 rotated. The node C i.e. 45 becomes the root of the tree with the node B(40) and A(50) as its left and right child.



4.3.Applications of AVL Trees

AVL trees are applied in the following situations:

- There are few insertion and deletion operations
- Short search time is needed
- Input data is sorted or nearly sorted
- AVL trees are mostly used for **in-memory sorts of sets and dictionaries**.
- AVL trees are also used extensively in database applications in which insertions and deletions are fewer but there are frequent lookups for data required.
- It is used in applications that require improved searching apart from the database applications
- AVL tree is a balanced binary search tree which employs rotation to maintain balance.
- It has application in story-line games as well.

5.B Tree

5.1.Introduction

B Tree is a specialized m-way tree that can be widely used for disk access. A B-Tree of order m can have at most m-1 keys and m children.

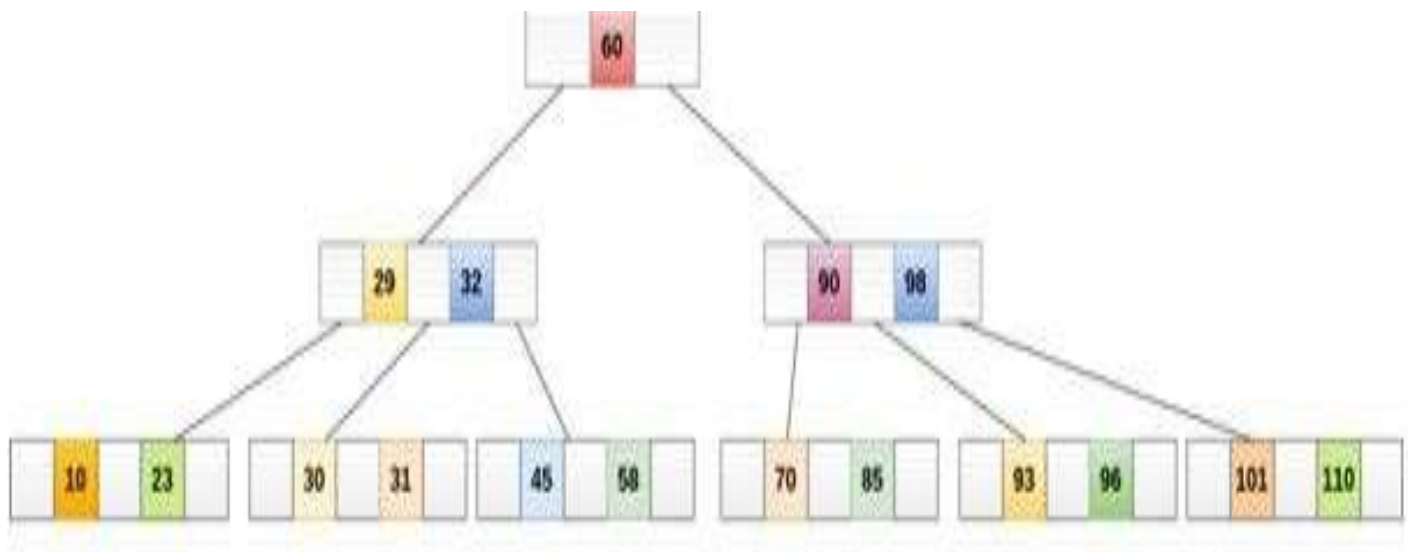
One of the main reasons for using B tree is its capability to store large number of keys in a single node and large key values by keeping the height of the tree relatively small.

A BTree of order m contains all the properties of an M way tree. In addition, it contains the following properties.

1. Every node in a B-Tree contains at most m children.
2. Every node in a B-Tree except the root node and the leaf node contain at least $m/2$ children.
3. The root nodes must have at least 2 nodes.
4. All leaf nodes must be at the same level.

It is not necessary that, all the nodes contain the same number of children but, each node must have $m/2$ number of nodes.

A B tree of order 4 is shown in the following image.



While performing some operations on B Tree, any property of B Tree may violate such as number of minimum children a node can have. To maintain the properties of B Tree, the tree may split or join.

5.2.Operations of B Trees

1. Creation

2. Insertion

3. Deletion

5.2.1. Creation :

Steps to Create a B-Tree

Step 1: Initialize the B-Tree

- Start by defining the order m of the B-Tree.
- Create an empty root node that will hold the first set of keys.

Step 2: Insert Keys into the B-Tree

- For each key to be inserted:
 1. Start at the Root:
 - Begin the insertion process from the root node.
 2. Locate the Appropriate Position:
 - Traverse the tree starting from the root to locate the appropriate leaf node where the key should be inserted.
 - Use the properties of a B-Tree to guide traversal:
 - If the key is less than the current key in the node, move to the left child.
 - If greater, move to the right child (or the appropriate middle child if there are multiple keys).

```

struct BTreeNode
{
    int val[MAX + 1], count;
    struct BTreeNode *link[MAX + 1];
};

Struct BTreeNode *root;
struct BTreeNode *createNode(int val, struct BTreeNode *child)
{
    struct BTreeNode *newNode;
    newNode = (struct BTreeNode *)malloc(sizeof(struct BTreeNode));
    newNode->val[1] = val;
    newNode->count = 1;
    newNode->link[0] = root;
    newNode->link[1] = child;
    return newNode;
}

```

5.2.2. Insertion

Insertions are done at the leaf node level. The following algorithm needs to be followed inorder to insert an item into B Tree.

1. Traverse the B Tree in order to find the appropriate leaf node at which the node can be inserted.
2. If the leaf node contain less than $m-1$ keys then insert the element in the increasing order.
3. Else, if the leaf node contains $m-1$ keys, then follow the following steps.
 - Insert the new element in the increasing order of elements.
 - Split the node into the two nodes at the median.
 - Push the median element upto its parent node.
 - If the parent node also contain $m-1$ number of keys, then split it too by following the same steps.

Example

Construct a **B-Tree of Order 3** by inserting numbers from 1 to 10.

Construct a B-Tree of order 3 by inserting numbers from 1 to 10.

insert(1)

Since '1' is the first element into the tree that is inserted into a new node. It acts as the root node.



insert(2)

Element '2' is added to existing leaf node. Here, we have only one node and that node acts as root and also leaf. This leaf node has an empty position. So, new element (2) can be inserted at that empty position.



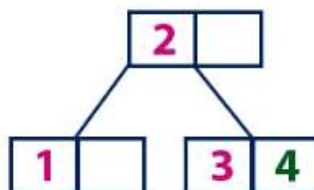
insert(3)

Element '3' is added to existing leaf node. Here, we have only one node and that node acts as root and also leaf. This leaf node doesn't have an empty position. So, we split that node by sending middle value (2) to its parent node. But here, this node doesn't have a parent. So, this middle value becomes a new root node for the tree.



insert(4)

Element '4' is larger than root node '2' and it is not a leaf node. So, we move to the right of '2'. We reach to a leaf node with value '3' and it has an empty position. So, new element (4) can be inserted at that empty position.



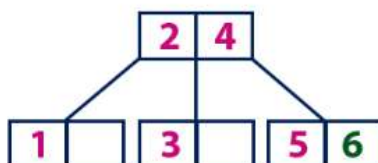
insert(5)

Element '5' is larger than root node '2' and it is not a leaf node. So, we move to the right of '2'. We reach to a leaf node and it is already full. So, we split that node by sending middle value (4) to its parent node (2). There is an empty position in its parent node. So, value '4' is added to node with value '2' and new element '5' added as new leaf node.



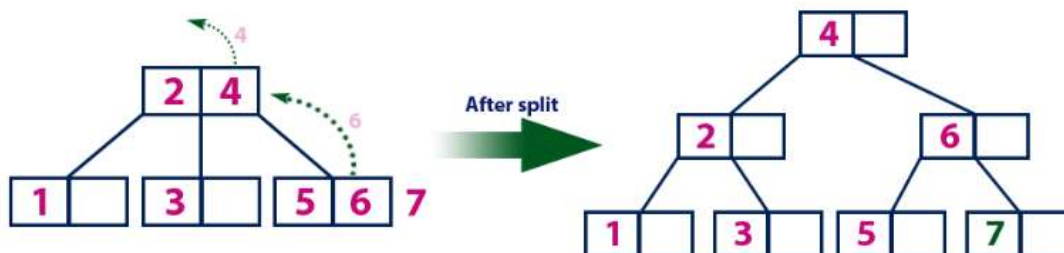
insert(6)

Element '6' is larger than root node '2' & '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a leaf node with value '5' and it has an empty position. So, new element (6) can be inserted at that empty position.



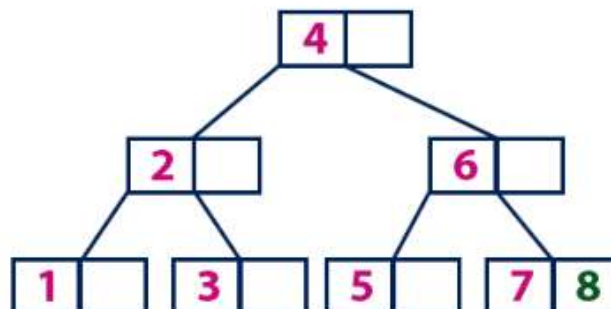
insert(7)

Element '7' is larger than root node '2' & '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a leaf node and it is already full. So, we split that node by sending middle value (6) to its parent node (2&4). But the parent (2&4) is also full. So, again we split the node (2&4) by sending middle value '4' to its parent but this node doesn't have parent. So, the element '4' becomes new root node for the tree.



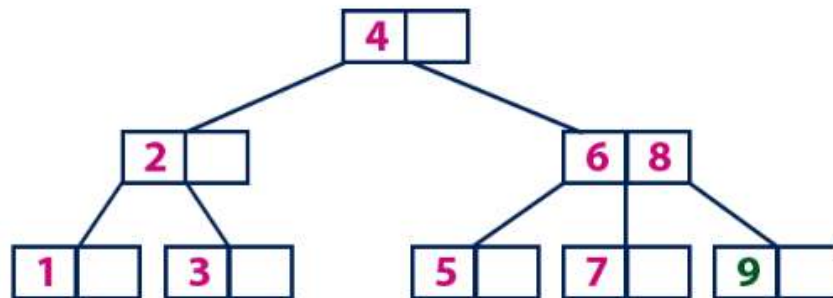
insert(8)

Element '8' is larger than root node '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a node with value '6'. '8' is larger than '6' and it is also not a leaf node. So, we move to the right of '6'. We reach to a leaf node (7) and it has an empty position. So, new element (8) can be inserted at that empty position.



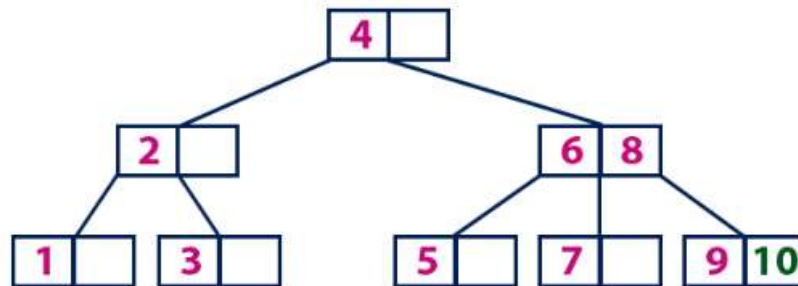
insert(9)

Element '9' is larger than root node '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a node with value '6'. '9' is larger than '6' and it is also not a leaf node. So, we move to the right of '6'. We reach to a leaf node (7 & 8). This leaf node is already full. So, we split this node by sending middle value (8) to its parent node. The parent node (6) has an empty position. So, '8' is added at that position. And new element is added as a new leaf node.



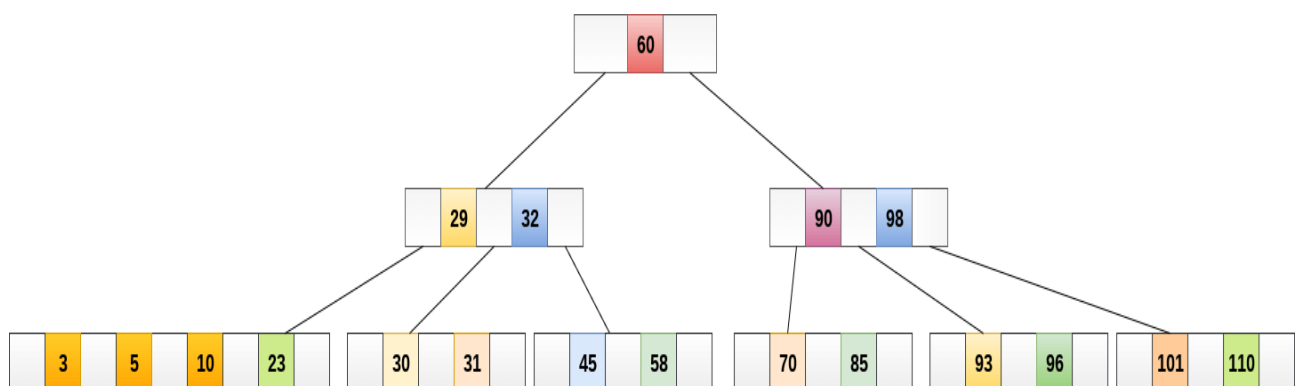
insert(10)

Element '10' is larger than root node '4' and it is not a leaf node. So, we move to the right of '4'. We reach to a node with values '6 & 8'. '10' is larger than '6 & 8' and it is also not a leaf node. So, we move to the right of '8'. We reach to a leaf node (9). This leaf node has an empty position. So, new element '10' is added at that empty position.

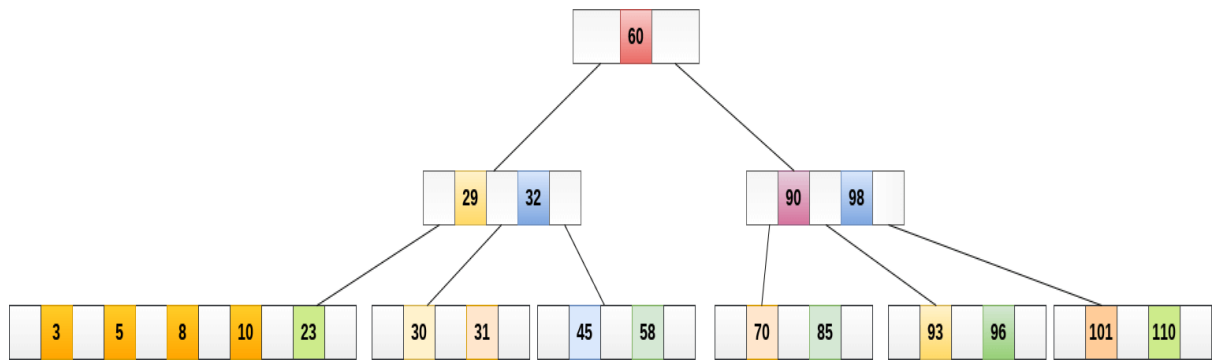


Example 2:

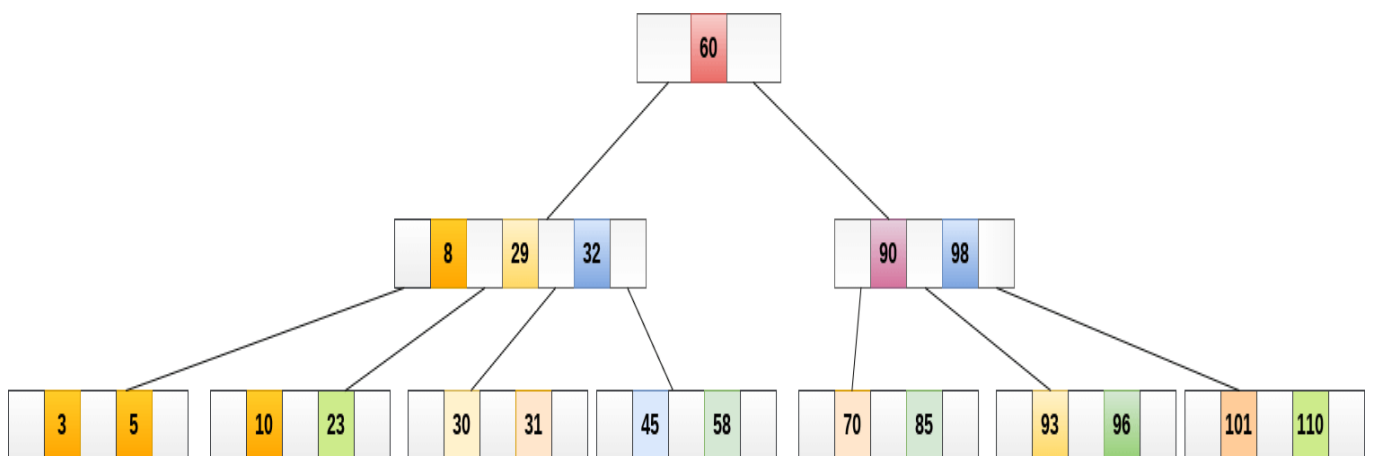
Insert the node 8 into the B Tree of order 5 shown in the following image.



8 will be inserted to the right of 5, therefore insert 8.



The node, now contain 5 keys which is greater than $(5 - 1 = 4)$ keys. Therefore split the node from the median i.e. 8 and push it up to its parent node shown as follows.



5.2.3. Deletion

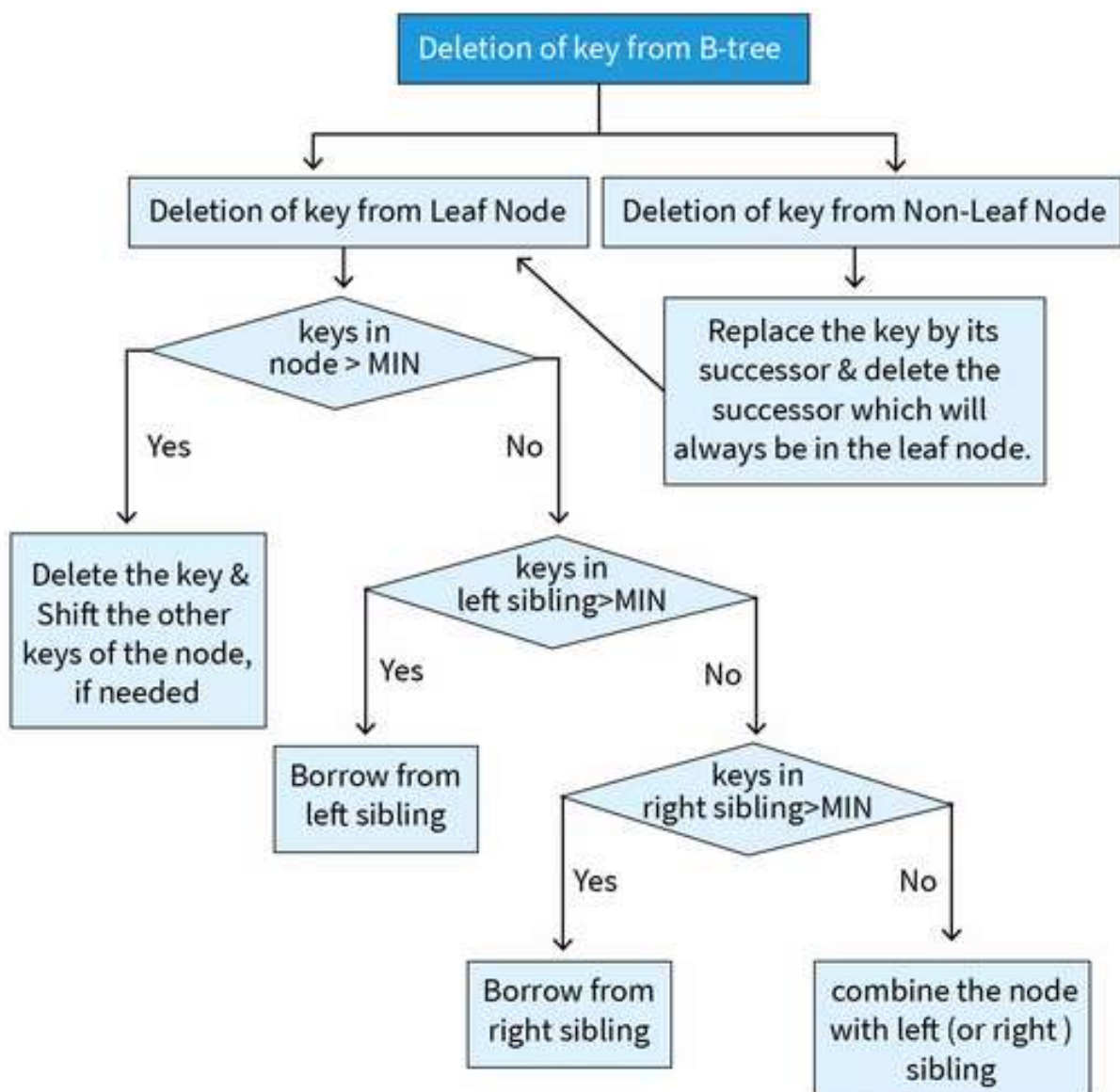
Deletion is also performed at the leaf nodes. The node which is to be deleted can either be a leaf node or an internal node. Following algorithm needs to be followed in order to delete a node from a B tree.

4. Locate the leaf node.
5. If there are more than $m/2$ keys in the leaf node then delete the desired key from the node.
6. If the leaf node doesn't contain $m/2$ keys then complete the keys by taking the element from right or left sibling.
 - If the left sibling contains more than $m/2$ elements then push its largest element up to its parent and move the intervening element down to the node where the key is deleted.
 - If the right sibling contains more than $m/2$ elements then push its smallest

element up to the parent and move intervening element down to the node where the key is deleted.

7. If neither of the sibling contain more than $m/2$ elements then create a new leaf node by joining two leaf nodes and the intervening element of the parent node.
8. If parent is left with less than $m/2$ nodes then, apply the above process on the parent too.

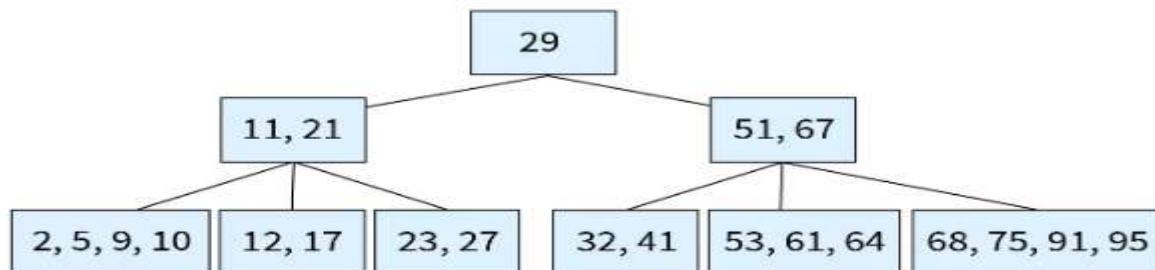
If the the node which is to be deleted is an internal node, then replace the node with its in- order successor or predecessor. Since, successor or predecessor will always be on the leaf node hence, the process will be similar as the node is being deleted from the leaf node.



Example 1

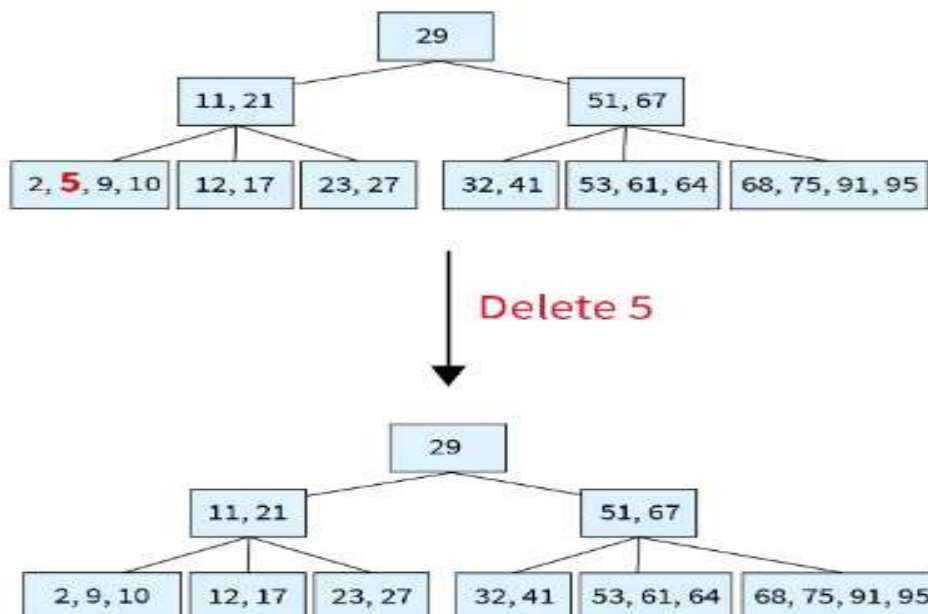
Now, let's understand deletion using an example. Consider the following B tree of order 5 with the nodes 5, 12, 32 and 53 to be deleted in the given order.

Since the order of the B tree is 5, the minimum and maximum number of keys in a node is 2 and 4 respectively.



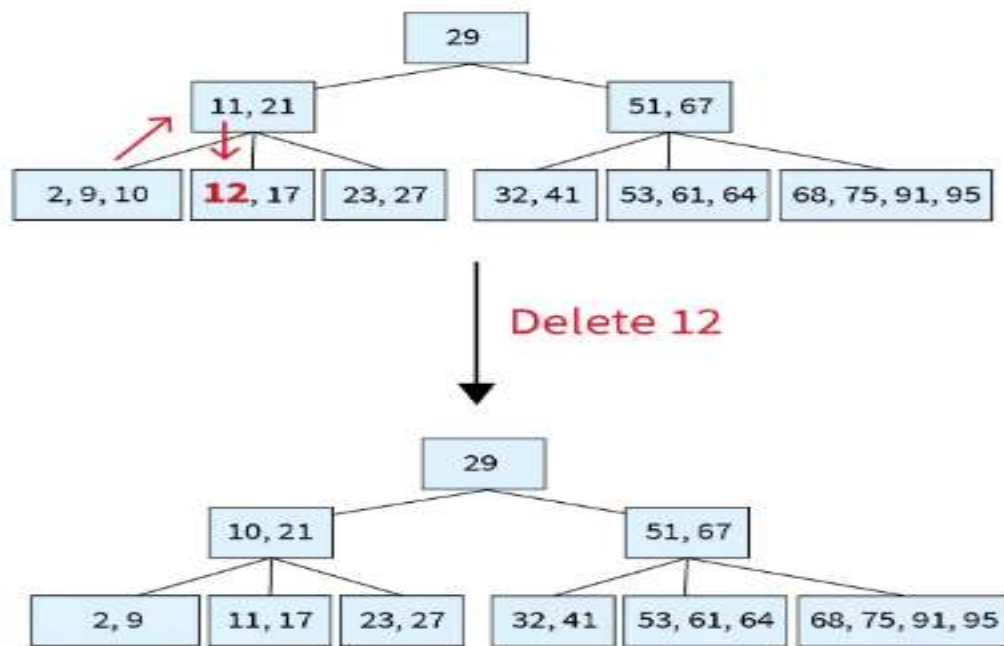
Step 1 - Deleting 5

Since 5 is a key in a leaf node with keys > MIN, this would be Case 1.1. A simple deletion with key shift would be done. Keys 9 and 10 are shifted left to fill the gap created by the deletion of 5.



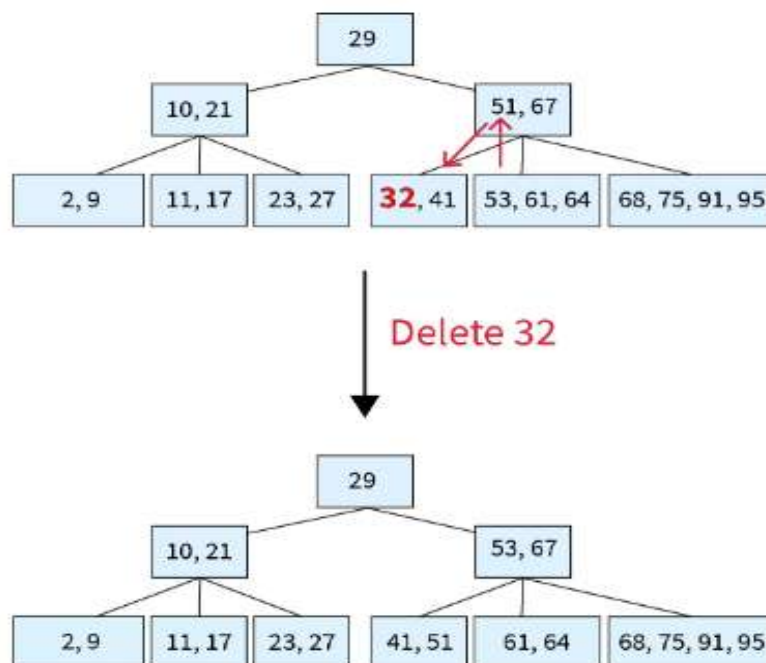
Step 2 - Deleting 12

Here, key 12 is to be deleted from node [12,17]. Since this node has only MIN keys, we will try to borrow from its left sibling [2,9,10] which has more than MIN keys. The parent of these nodes [11,21] contains the separator key 11. So, the last key of left sibling (10) is moved to the place of the separator key and the separator key is moved to the underflow node (the node where deletion took place). The resulting tree after deletion can be found as follows:



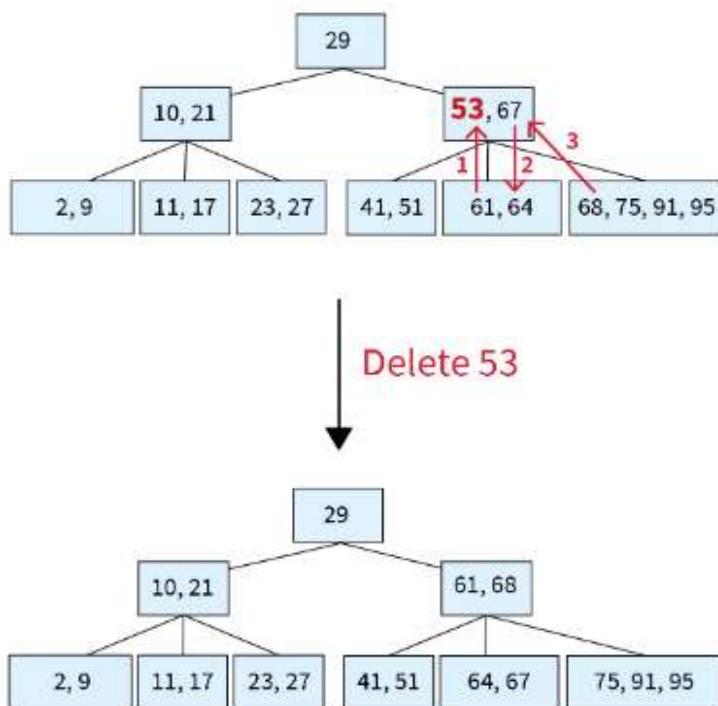
Step 3 - Deleting 32

Here, key 32 is to be deleted from node [32, 41]. Since this node has only MIN keys and does not have a left sibling, we will try to borrow from its right sibling [53, 61, 64] which has more than MIN keys. The parent of these nodes [51, 67] contains the separator key 51. So, the first key of right sibling (53) is moved to the place of the separator key and the separator key is moved to the underflow node (the node where deletion took place). The resulting tree after deletion can be found as follows:



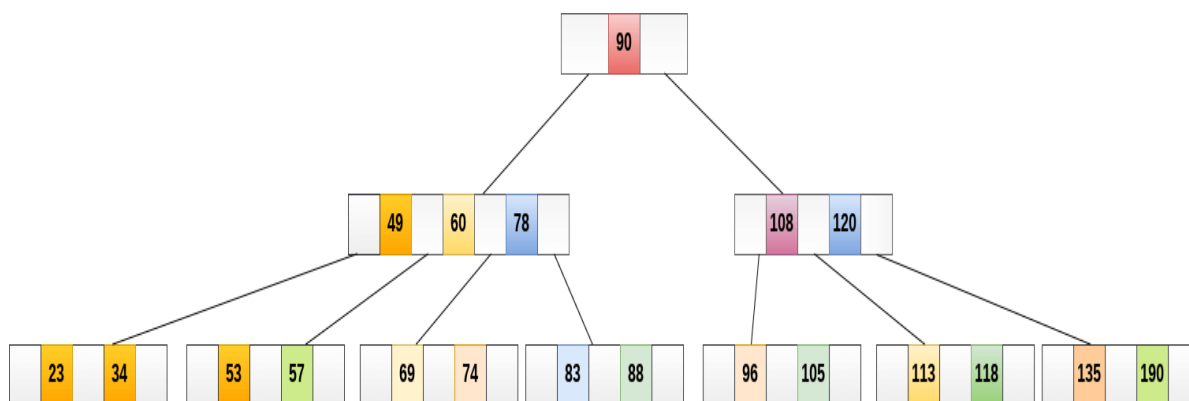
Step 4 - Deleting 53

Here key 53 is to be deleted from node [53, 67] which is a non-leaf node. In such a case, the successor key (61) will be copied in place of 53 and now the task reduces to deletion of 61 from the leaf node. Since this node would have less than MIN keys, we check for the left sibling. Since the left sibling has only MIN keys, we move to the right sibling. The leftmost key of the right sibling (68) moves to the parent node and replaces the separator (67) while the separator shifts to the underflow node making it [64,67]. The resulting tree after deletion can be found as follows:

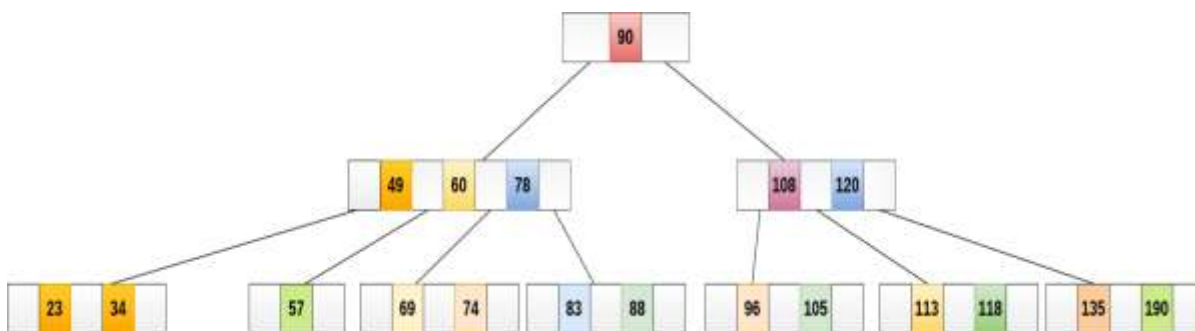


Example 2

Delete the node 53 from the B Tree of order 5 shown in the following figure.

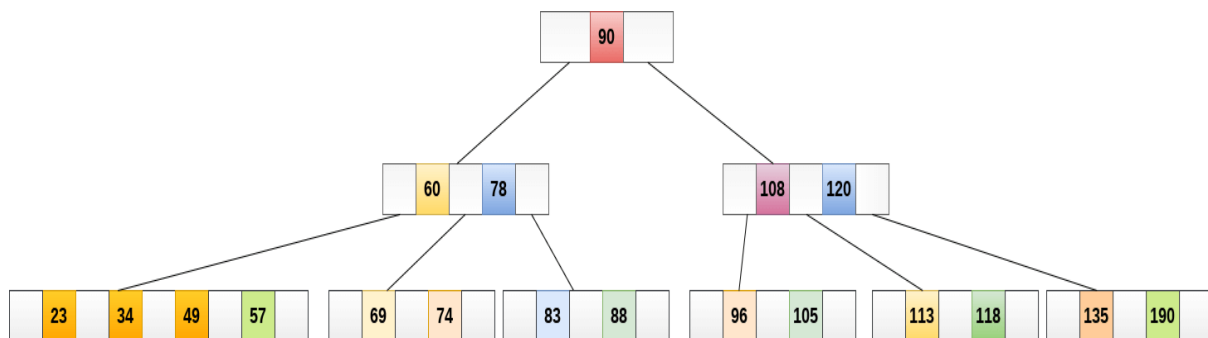


53 is present in the right child of element 49. Delete it.



Now, 57 is the only element which is left in the node, the minimum number of elements that must be present in a B tree of order 5, is 2. it is less than that, the elements in its left and right sub-tree are also not sufficient therefore, merge it with the left sibling and intervening element of parent i.e. 49.

The final B tree is shown as follows.



5.3.Application of B-Tree:

B-trees are commonly used in applications where large amounts of data need to be stored and retrieved efficiently. Some of the specific applications of B-trees include:

- **Databases:** B-trees are widely used in databases to store indexes that allow for efficient searching and retrieval of data.
- **File systems:** B-trees are used in file systems to organize and store files efficiently.
- **Operating systems:** B-trees are used in operating systems to manage memory efficiently.
- **Network routers:** B-trees are used in network routers to efficiently route packets through the network.
- **DNS servers:** B-trees are used in Domain Name System (DNS) servers to store and retrieve information about domain names.
- **Compiler symbol tables:** B-trees are used in compilers to store symbol tables that allow for efficient compilation of code.